WPT Day 9 — MVC Architecture, Controllers, Routers & Login Authentication

WPT Day 9 (29 October) · Express.js + MySQL · written so that someone who missed the lecture can follow it from zero: vocabulary first, then the working code, the refactor, the request flow, and authentication.

How to Read These Notes

The lecture follows a story, and so do these notes. **Part 1** gives you the vocabulary, **Part 2** shows a working one-file application, and **Part 3** explains why "working" is not "well designed". **Parts 4–6** rebuild it into layers without changing a single URL. **Parts 7–9** explain how a request travels through those layers, why APIs need authentication, and how the admin registration API was written. **Parts 10–11** are revision material and a run-book.

Code that came from the lecture appears **verbatim**, including the instructor's comments. Long lines are only wrapped so they fit the page; no token is changed. Where the lecture only *described* something (a model class, service/DAO layers, token creation, password hashing), the example is inside a block marked **"Added in these notes"** so you always know what is lecture content and what is supporting material.

Two warnings before you start:

- Run the archive exactly as it is: the config file is `src/configs/DbConfig.js` with a capital **D** and **C**. The lecture says "dbconfig" out loud; on Linux the wrong casing is a hard error.
- `node index.js` does not watch files. Restart the server after every change.

Table of Contents

1. Ground Zero: The Vocabulary You Need
2. The Starting Point: One File That Works
3. MVC: The Blueprint for Splitting the App
4. The Controller Layer
5. The Config Layer
6. The Router Layer and the `export default` Trap
7. The Architecture in Motion: Flow, Middleware, Service, DAO
8. Securing the APIs: Login Authentication
9. Admin Registration and the Password Loophole
10. Revision: Tables, Q&A, Practice
11. Run-Book: Commands, Postman, Endpoints

1. Ground Zero: The Vocabulary You Need

If you have written JavaScript but never a backend, this section is the difference between copying code and understanding it. Everything here is used again later, so it is worth ten minutes.

1.1 Client, server, request, response

A **client** is any program that asks for data: a browser, Postman, a React app, an Android app. A **server** is a program that waits for those requests and answers them. The client sends a **request**; the server sends a **response**.

```
CLIENT                                          SERVER
browser / Postman / React  ——— request ———→  Express app (port 7500)
browser / Postman / React  ←——— response ———  JSON + status code
```

Your Express application *is* the server. It runs continuously on a port (7500 in this project) and handles one request after another.

1.2 The anatomy of an HTTP request

Every request has four parts. You will use all four today:

```
GET /product/2 HTTP/1.1
Host: localhost:7500
Content-Type: application/json

{ "name": "Keyboard" }
```

Part	Example	Where Express puts it
Method	`GET`, `POST`, `DELETE`	decides which route runs
URL / path	`/product/2`	`request.params`, `request.query`
Headers	`Content-Type: application/json`	`request.headers`
Body	the JSON object	`request.body` (only after parsing)

A response has a **status code**, headers, and a body:

```
HTTP/1.1 200 OK
Content-Type: application/json

{ "id": 2, "name": "Keyboard", "price": 6677 }
```

1.3 Methods and status codes

The method describes the **intent** of the request. The four used in this project:

Method	Meaning	Example
`GET`	read something	`GET /product`
`POST`	create something	`POST /product`
`PUT` / `PATCH`	update something (not written today)	`PUT /product/2`
`DELETE`	remove something	`DELETE /product/2`

The status code is a three-digit summary of what happened:

Code	Name	Used when
`200`	OK	normal success
`201`	Created	a new resource was created (this project returns `200`)
`400`	Bad Request	the client sent invalid data
`401`	Unauthorized	not logged in / token missing (authentication)
`403`	Forbidden	logged in but not allowed (authorization)
`404`	Not Found	the URL or the requested row does not exist
`500`	Internal Server Error	the server itself failed (database down, bug)

The difference between `404` and `500` matters: asking for `product/999` is a **valid request for something that does not exist** (`404`); a broken SQL query is **the server's fault** (`500`). Never return `500` for "not found", and never leak the stack trace to the client.

1.4 JSON and endpoints

JSON (JavaScript Object Notation) is the text format both sides speak. An object uses `{ }` and keys in double quotes; an array uses `[ ]`.

```
{ "name": "Keyboard", "price": 6677, "qty": 8 }
```

An **endpoint** is one method + one URL: `POST /product` and `GET /product` are two different endpoints that share a URL. In Express each endpoint is written as `app.get( ... )` or `router.post( ... )`.

1.5 Node, npm, Express, modules

- **Node.js** runs JavaScript outside the browser. It gives you `http`, file access, and the module system.
- **npm** downloads libraries into `node_modules/` and records them in `package.json`.
- **Express** is a small library on top of Node that turns a raw HTTP server into readable code: `app.get(path, handler)` instead of parsing raw sockets.

The project's dependencies:

Package	Why
`express`	routing, middleware, `request` / `response` helpers
`mysql2`	talk to MySQL, with promises (`mysql2/promise`)

A **module** is one file. Nothing in a file is visible to another file until it is **exported**:

```
// math.js
export function add(a, b) { return a + b; }

// index.js
import { add } from "./math.js";
```

There are two export styles, and mixing them up is the single most common error of the day:

Style	Export	Import
Named	`export function save() {}`	`import { save } from "./x.js"`
Default	`export default router`	`import router from "./x.js"`

The project sets `"type": "module"` in `package.json`. That enables this modern syntax (ESM) and requires the `.js` **extension** in every import path. Without it, Node expects the old `require()` style.

1.6 The MySQL pieces you will see

Term	Meaning
Table	like a spreadsheet: `product`, `admin`
Row	one record (one product)

Term	Meaning
Column	one field (`name` , `price`)
Primary key	the unique identifier of a row (`id`)
`AUTO_INCREMENT`	MySQL fills `id` automatically: 1, 2, 3…
`VARCHAR(n)`	text up to `n` characters
Query	SQL sent to the database (`SELECT` , `INSERT` , `DELETE`)
Placeholder `?`	a slot filled safely by the driver, instead of gluing strings

A **connection** is an open conversation with MySQL, and opening one is slow. A **pool** keeps several connections open and lends them out, which is why the project creates a pool once instead of connecting per request.

1.7 `async` / `await` in one paragraph

Talking to a database takes time. Node does not sit and wait; it continues with other work and comes back when the result is ready. `await` marks the point where the function pauses until the promise resolves, and `async` declares that a function is allowed to `await` . Every controller that touches MySQL is therefore `async` , and every query is `await` ed. Forgetting `async` is a syntax error, not a style issue.

---

2. The Starting Point: One File That Works

The lecture begins with a **working** application, not a broken one. On Day 8 the whole product CRUD API lived in one `index.js` : connection pool, routes, SQL and responses together. It runs perfectly — and that is the trap.

2.1 What the application does

Method	URL	Job
`GET`	`/`	welcome message
`POST`	`/product`	insert a product
`GET`	`/product`	fetch all products
`GET`	`/product/:id`	fetch one product

Method	URL	Job
`DELETE`	`/product/:id`	delete one product

2.2 The complete code — `index.js`, before refactoring

> **Note:** the single-file version was shown on screen in the previous session rather than typed out again. The listing below is **reconstructed** from the refactored files in the class archive; the SQL, variable names, responses and route paths are identical, only the file boundaries differ.

```
import express from "express";
import { createPool } from "mysql2/promise";

const app = express();
app.use(express.json());

// ———————— 1. database configuration + pool ————————
const sqlConnectionPool = createPool({
    host: "localhost",
    port: 3306,
    user: "root",
    password: "root",
    database: "productdb",
    connectionLimit: 10,
    queueLimit: 0
});

// ———————— 2. welcome route ————————
app.get("/", (request, response) => {
    response.send({ message: "Welcome to crud APIs" });
});

// ———————— 3. insert a product ————————
app.post("/product", async (request, response) => {
    try {
        const { name, price, description, qty } = request.body;

        const insertQry =
            `INSERT into product(name, price, description, qty) VALUES(?,?,?,?)`;

        const [result] = await sqlConnectionPool.query(
            insertQry,
            [name, price, description, qty]
        );

        response.send({
            message: "product inserted",
```

```
            insertedId: result.insertId
        });
    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
});

// ———————— 4. fetch all products ————————
app.get("/product", async (request, response) => {
    try {
        const fetchAllProductsQry = "SELECT * FROM product";

        const [result] = await sqlConnectionPool.query(fetchAllProductsQry);

        response.send(result);
    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
});

// ———————— 5. fetch one product ————————
app.get("/product/:id", async (request, response) => {
    try {
        const { id } = request.params;

        const fetchByIdQuery = "SELECT * FROM product WHERE id = ?";

        const [result] = await sqlConnectionPool.query(fetchByIdQuery, [id]);

        if (result.length > 0) {
            response.send(result[0]);
        } else {
            response.status(404).send({
                message: `Product with id ${id} not found`
            });
        }
    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
});

// ———————— 6. delete one product ————————
app.delete("/product/:id", async (request, response) => {
    try {
        const { id } = request.params;

        const deleteQry = "DELETE FROM product WHERE id = ?";
```

```
        const [result] = await sqlConnectionPool.query(deleteQry, [id]);

        response.send({ message: "Product Deleted" });
    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
});

app.listen(7500);
```

2.3 Line by line

Lines 1–2 — imports. `express` creates the server. `createPool` comes from `mysql2/promise`, the promise-based wrapper that lets you `await` queries.

Line 4 — the application object. `express()` returns `app`, the object on which every route is registered.

Line 5 — the body parser. `express.json()` is middleware: it reads a JSON request body and puts the parsed object on `request.body`. Without it, `request.body` is `undefined`, and destructuring `const { name } = request.body` throws a `TypeError`.

Lines 8–16 — the pool. This block is **configuration**. It has nothing to do with products, customers or any endpoint; it only says *how to reach MySQL*. `connectionLimit: 10` means at most ten connections are open at once; `queueLimit: 0` means extra requests wait in an unlimited queue instead of being rejected.

Lines 19–21 — the welcome route. A controller function with no name: it takes `request` and `response` and sends a fixed JSON message.

Lines 24–42 — insert. Destructure the body, build a **parameterised** query (`?` placeholders), pass the values as an array. `query()` resolves to `[rows, fields]`; for an `INSERT`, `rows` is a result object whose `insertId` is the new primary key.

Lines 45–57 — fetch all. One `SELECT * FROM product`, and the rows array is sent directly. An empty table gives `[]` with status `200`, not `404`.

Lines 60–75 — fetch one. `request.params.id` comes from the `:id` segment and is always a **string**; MySQL coerces it for `WHERE id = ?`. `result` is an array — empty if nothing matched — so `result.length > 0` chooses between the row and a `404`.

Lines 78–91 — delete. Same parameterised style. MySQL returns `affectedRows`, but the lecture does not branch on it, so deleting a non-existent id still answers `{"message":"Product Deleted"}`.

Line 93 — start. `app.listen(7500)` binds the server to port 7500, used in every Postman test today.

2.4 Running it

```
node index.js
```

`POST http://localhost:7500/product`, body → raw → JSON:

```
{
  "name": "Keyboard",
  "price": 6677,
  "description": "Mechanical keyboard",
  "qty": 8
}
```

```
{ "message": "product inserted", "insertedId": 1 }
```

`GET http://localhost:7500/product`:

```
[
  {
    "id": 1,
    "name": "Keyboard",
    "price": 6677,
    "description": "Mechanical keyboard",
    "qty": 8
  }
]
```

Everything works. **This is the trap.**

2.5 Phase 2 — what is actually wrong with it?

Beginners often answer "it will not run". It runs. The instructor is precise:

> "Kya code chalega nahi? Naya error aaega? Nahi, code is going to work totally fine. Your API is going to work totally fine. Lekin problem kahan pe hai? Problem aane wali hai in the **scalability** and in the **maintainability** of your application."

1. An enterprise app has hundreds of APIs, not five. Customer, order, cart, admin, payment — every entity appends its routes and logic to the same file. A file that is pleasant at 5 endpoints becomes unreadable at 200.

2. Four unrelated concerns are mixed. Configuration (host, user, password), routing (which URL), request/response handling (read body, send JSON), and business logic + SQL (validation, `INSERT`, `SELECT`) all live in the same file, interleaved.

3. Merge conflicts. Two developers working on "product delete" and "product insert" edit the same file. A bad merge can break an API nobody touched.

4. Debugging is slower. A stack trace points at `index.js:487`; you scroll through hundreds of unrelated lines to find the failing function.

5. Nothing can be reused. The insert SQL is inside a route handler. A second feature that needs the same insert copies it, and now two copies can drift apart.

6. Testing is coarse. You cannot load one function in isolation: importing the file creates a pool and starts a server.

> **Pitfall:** "It runs" is not a design argument. The cost of a monolith is paid later, by every future change.

2.6 The house analogy

> "You have a separate **kitchen** where all utensils and kitchen-related items are kept. You have a certain **wardrobe** where you keep only your clothes. There is a separate **parking** where only your vehicles stay. We have used that segregation — separation of concerns — into every domain."

House	Backend
Kitchen — utensils, stove, groceries	`configs/` — database settings
Wardrobe — only clothes	`controllers/` — only request/response functions
Parking — only vehicles	`routers/` — only URL → controller mapping
Main door — you enter, then go to a room	`index.js` — receives every request, delegates it

Nobody builds a house with the fridge, the bed and the motorbike in one room. Nobody should build a server with the pool, the routes and the SQL in one file.

2.7 What changes and what must not

Refactoring is **behaviour-preserving**. The client must not notice that anything happened.

```
BEFORE
client ──► index.js ──► MySQL
           (pool + routes + SQL + responses all in one file)

AFTER
client ──► index.js ──► router ──► controller ──► MySQL
```

In the new layout `index.js` starts the server, the router chooses the handler, the controller sends the response, and the config file owns the connection pool.

Aspect	Changes?	Why
URLs, JSON shape, status codes, port 7500	**No**	they are the contract with the client
File layout	**Yes**	code moves into `src/` sub-folders
`index.js` size	**Yes**	shrinks to about a dozen lines
Imports/exports	**Yes**	every moved function is exported and imported

> **Exam:** "Refactoring into layers is a behaviour-preserving change: the API contract stays identical while the code becomes modular, so each layer can be changed, tested and scaled independently."

2.8 Express vs NestJS

> "In Express.js we have to do this thing **manually**. But if you use **Nest.js**, out of the box it is a framework that provides you a complete ecosystem for separation of concerns... But in Express.js, we have to create this entire thing manually from scratch."

	Express.js	NestJS
Type	minimal web framework	opinionated, full-featured framework
Structure	**you** design folders and layers	prescribed (modules, controllers, providers)
Learning curve	low	higher for a beginner
Flexibility	total freedom — including freedom to make a mess	guard rails and conventions
Dependency injection	manual	built in

	Express.js	NestJS
Today	used here	the next step to explore

You build it by hand first so that NestJS's generated folders make sense later.

---

3. MVC: The Blueprint for Splitting the App

3.1 What a layer is

A **layer** is a group of files that all do the same *kind* of job. A layer has a narrow contract: it receives certain inputs, returns certain outputs, and does not know (or care) about the internals of the other layers. That ignorance is the point — it lets you replace one layer without touching the rest.

MVC (Model–View–Controller) is the classic design pattern for this split. The instructor compares it to a house plan (BHK): separate sections for the bedroom, the hall and the kitchen.

3.2 The three parts

Part	Meaning	In this project
Model	represents an entity and its properties	discussed; `src/models/` is not created (raw SQL is used)
View	what the user sees — the UI	deferred to React
Controller	function that accepts the request and generates the response	`src/controllers/`

E-commerce entities are **Customer, Product, Order, Cart**. A model documents the shape of an entity — its fields, types and rules — so that every developer knows what a "product" is without reading every query. An ORM such as Sequelize or Prisma generates these from a schema.

Added in these notes — the simplest honest model in plain JavaScript (the lecture discusses models but does not create the folder):

```
// src/models/Product.js
export class Product {
    constructor({ name, price, description, qty }) {
        this.name = name;
        this.price = price;
        this.description = description;
        this.qty = qty;
```

```
    }

    isValid() {
        return Boolean(this.name)
            && Number(this.price) > 0
            && Number(this.qty) ≥ 0;
    }
}
```

The backend built today needs only **Model + Controller**; the **View** arrives with the React application. The controller returns JSON, and React draws the screen.

> **Exam:** a controller is **only** a `(request, response)` function. It must not contain business rules and must not know that a database exists.

3.3 The Single Responsibility Principle

One file, one job. If you cannot describe a file's purpose in one sentence **without the word "and"**, it is doing too much.

File	Its one job
`index.js`	create the app, register middleware and routers, listen
`routers/*.js`	map method + URL to a controller
`controllers/*.js`	accept the request, send the response
`configs/*.js`	hold configuration (database, mail, keys)

3.4 The target folder tree

```
crud-apis/
├── index.js                ← entry point: start the server
├── package.json            ← "type": "module", dependencies
└── src/
    ├── configs/
    │   └── DbConfig.js     ← pool / DB settings
    ├── controllers/
    │   ├── WelcomeController.js
    │   ├── ProductController.js
    │   └── AdminController.js
    ├── middlewares/        ← created today, filled in the evening session
    └── routers/
        ├── WelcomeRouter.js
        ├── ProductRouter.js
        └── AdminRouter.js
```

Create them once:

```
mkdir src
mkdir src/configs src/controllers src/middlewares src/routers
```

3.5 Before and after

Criterion	One file	MVC + layers
Files	1	many
Find the insert logic	scroll	open `ProductController.js`
Add a customer API	append to `index.js`	new controller + router + one `app.use` line
Change DB credentials	edit the route file	edit `DbConfig.js`
Two devs, two entities	merge conflicts	separate files
Test one handler	hard (the file starts a server)	import the function
API behaviour	correct	**identical**

The cost is more files, more imports, and more chance of a wrong path. The lecture hits exactly that bug in Part 9 — and the architecture is still worth it.

---

4. The Controller Layer

Now the refactor begins for real. The instructor moves **only the request/response functions** out of `index.js`, one entity at a time.

> "All those functions which are accepting the request and generating the response — in `app.get` or `app.post`, the function which has the `request` and `response` argument — that is nothing, just termed as a **controller function**. So these functions, don't define them in the index file. Create a separate layer in the application that will be the controller."

4.1 One controller file per entity

`ProductController.js` holds every product request/response function. A customer would get `CustomerController.js`; admin gets `AdminController.js`. This is the same rule as database tables: one table per entity, one file per entity.

The base URL `/` has nothing to do with products, so it gets its own **welcome controller**.

4.2 `WelcomeController.js`

```
// src/controllers/WelcomeController.js
export const welcome = (request, response) => {
    response.send({ message: "Welcome to crud APIs" });
}
```

Line by line: `export` makes the function visible outside the file; without it the function is private to that module. `const welcome` is a **named** export. The arrow function receives the two objects Express always passes.

The file is already called `WelcomeController.js`, so the function is simply `welcome` — repeating "controller" inside it adds nothing.

4.3 Wiring it into `index.js`

```
import express from "express";
import { welcome } from "./src/controllers/WelcomeController.js";

const app = express();
app.use(express.json());

app.get("/", welcome);      // pass the REFERENCE — do not call it

app.listen(7500);
```

`import { welcome }` uses braces because the export was named. The path starts with `./` because `index.js` sits in the project root, and ends with `.js` because ESM requires the extension.

> **Pitfall:** `app.get("/", welcome())` **calls** the function during startup. `request` is `undefined`, so it throws `TypeError: Cannot read properties of undefined (reading 'send')`. Express must receive the function itself, and will call it once per request.

That one line is the whole mental model for the refactor: **`index.js` is now a receptionist.** It receives every visitor and forwards them to the right department; it no longer does the department's work.

4.4 `ProductController.js` — the complete file

```
import { sqlConnectionPool } from '../configs/DbConfig.js';

export async function save(request, response) {
    try {
        // step 1: get data from client
        // {name:"", price: 6677, description:"", qty:8}
        const { name, price, description, qty } = request.body;

        // step 2: create a SQL INSERT query
        // const insertQry = `INSERT into product(name, price, description, qty)
        //   VALUES('${name}', ${price}, '${description}', ${qty})`
        // this above approach (concating data directly in the SQL query)
        // is prone to sql injection attack

        const insertQry =
            `INSERT into product(name, price, description, qty) VALUES(?,?,?,?)`;

        // step 3: run this query in the db
        const [result] = await sqlConnectionPool.query(
            insertQry,
            [name, price, description, qty]
        );
        console.log(result.insertId);

        response.send({
            message: "product inserted",
            insertedId: result.insertId
        });

    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
}

export async function getAll(request, response) {
    try {

        // step 1: create a select query

        const fetchAllProductsQry = "SELECT * FROM product";

        // step 2: run this query
        const [result] = await sqlConnectionPool.query(fetchAllProductsQry);

        response.send(result);
```

```
    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
}

export async function getById(request, response) {
    try {

        // step 1: read id from url parameter

        const { id } = request.params;

        // step 2: create a select query

        const fetchByIdQuery = "SELECT * FROM product WHERE id = ?";

        // step 3: run this query

        const [result] = await sqlConnectionPool.query(fetchByIdQuery, [id] );

        if(result.length > 0){
            response.send(result[0]);
        }
        else{
            response.status(404).send({
                message: `Product with id ${id} not found`
            });
        }

    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
}

export async function deleteById(request, response){
    try {

        // step 1: read id from url parameter

        const { id } = request.params;

        // step 2: create a delete query

        const deleteQry = "DELETE FROM product WHERE id = ?";

        // step 3: run this query

        const [result] = await sqlConnectionPool.query(deleteQry, [id]);
        console.log(result);
```

```
        response.send({message:"Product Deleted"});


    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
}
```

Notice the commented-out line in `save`. It is kept on purpose: it is the dangerous version. Gluing `'${name}'` into the SQL string makes user input **code**. If someone sends `name = "x'); DROP TABLE product; --"`, the database executes it. The `?` placeholders keep every value as **data**, no matter what characters it contains.

4.5 Walkthrough: `save`

Step	Code	What happens
1	`const { name, price, description, qty } = request.body`	reads the four fields; works only because `express.json()` ran first
2	`` `… VALUES(?,?,?,?)` ``	builds the statement with four slots
3	`await sqlConnectionPool.query(insertQry, [name, price, description, qty])`	sends the statement and the values separately
4	`const [result] = …`	`query()` resolves to `[rows, fields]`; only the first element is needed
5	`result.insertId`	the `AUTO_INCREMENT` id MySQL assigned
6	`response.send({ … })`	sends JSON with status `200`
7	`catch`	logs the real error for the developer, returns a generic message to the client

`async` is not optional: `await` is only legal inside an `async` function. Removing it gives `SyntaxError: await is only valid in async functions`.

Output: `POST /product` with `{"name":"Mouse","price":499,"description":"Wireless","qty":20}` → `{"message":"product inserted","insertedId":2}`.

4.6 The other three handlers

`getAll` uses no request data, but `request` still exists in the signature because Express calls `handler(request, response, next)`; if you declare only one parameter, that parameter **is** the request, not the response. It returns an array — `[]` when the table is empty, with status `200`.

`getById` reads `request.params.id`, which is filled from the route pattern `/product/:id` and is **always a string**. `SELECT` always returns an array, so "not found" is an **empty array**, not an error. `result.length > 0` picks the row or the `404`.

`deleteById` logs the result object (`affectedRows`, etc.) but does not branch on it, so deleting a non-existent id still answers `{"message":"Product Deleted"}`. A production API would check `result.affectedRows === 0` and return `404`; that is a deliberate simplification for today.

> **Pitfall:** `result` from a `SELECT` is always an array, even when one row matched. `result.name` is `undefined`; you need `result[0].name`. For `INSERT` / `DELETE`, `result` is a result object with `insertId` / `affectedRows`.

4.7 The naming convention

Inside `ProductController.js`, the entity is already established, so the functions are `save`, `getAll`, `getById`, `deleteById` — not `saveProduct`, `getAllProducts`. The instructor uses the database as the analogy:

> "In your database, if one table is `product`, then what will be the column name? Kya usmein `product_id`, `product_name`, `product_price` likhne ki zarurat hai? Nahi. We can only give `id`, `name`... It is understood that `id` is of the product."

Redundant	Clean	Reason
`saveProduct`	`save`	the file already says Product
`getAllProducts`	`getAll`	same
`getProductById`	`getById`	same
`product_name` (column)	`name`	the table already says product

The rule is: **do not repeat context that the file already provides.** When a function serves many entities from one shared file, the prefix becomes necessary: `saveProduct`, `saveCustomer`, `saveOrder` in a single `persistence.js`.

4.8 Importing several functions

```
import {
    save, getAll, getById, deleteById
} from "./src/controllers/ProductController.js";
```

After the move, `index.js` contains no SQL and no business logic — only URL → function mapping:

```
app.get("/", welcome);
app.post("/product", save);
app.get("/product", getAll);
app.get("/product/:id", getById);
app.delete("/product/:id", deleteById);
```

Method	URL	File	Function
GET	`/`	`WelcomeController.js`	`welcome`
POST	`/product`	`ProductController.js`	`save`
GET	`/product`	`ProductController.js`	`getAll`
GET	`/product/:id`	`ProductController.js`	`getById`
DELETE	`/product/:id`	`ProductController.js`	`deleteById`

Pitfall: named imports are case-sensitive and must match the exported identifier. `import { getall }` fails with *"does not provide an export named 'getall'"*. A missing `.js` or a wrong `./` vs `../` gives `ERR_MODULE_NOT_FOUND`. Ctrl+click the path in VS Code: it must jump to the real file.

4.9 Errors from this step

Error	Cause	Fix
`ERR_MODULE_NOT_FOUND`	missing `.js`, wrong depth, wrong casing	correct the path
`welcome is not a function`	wrong export, or nothing exported	check the `export` keyword

Error	Cause	Fix
`Cannot read properties of undefined (reading 'send')`	wrote `welcome()`	pass `welcome`
`await is only valid in async functions`	missing `async`	add it
`Cannot destructure property 'name' of 'request.body'`	parser missing / body not JSON	register `express.json()`; Postman raw → JSON
`sqlConnectionPool is not defined`	missing import in the controller	import from `../configs/DbConfig.js`

Verify nothing changed: `GET /` → welcome JSON; `POST /product` → `insertedId`; `GET /product` → array; `GET /product/999` → `404`. If any response changed, the refactor introduced a bug.

---

5. The Config Layer

The pool is neither business logic nor request handling. It is **configuration**, and configuration belongs in `src/configs/`.

5.1 What a connection pool is

Opening a MySQL connection is expensive: a TCP handshake, authentication, session setup. If every request opened and closed one, the database would spend its time saying hello.

A **pool** is a set of reusable connections. A request borrows one, runs its query, and returns it. `connectionLimit: 10` caps the set; if all ten are busy, new queries wait in a queue.

```
WITHOUT A POOL                WITH A POOL
connect → query → close       borrow → query → return
(repeat for every request)    (10 connections reused)
```

5.2 `DbConfig.js` — complete

```
// src/configs/DbConfig.js
import { createPool } from "mysql2/promise";

export const sqlConnectionPool = createPool({
    host: "localhost",
    port: 3306,
```

```
    user: "root",
    password: "root",
    database: "productdb",
    connectionLimit: 10,
    queueLimit: 0
});
```

Option	Meaning
`host`	where MySQL runs (`localhost` for this course)
`port`	MySQL's port, `3306` by default
`user` / `password`	database credentials
`database`	which schema to use (`productdb`)
`connectionLimit`	maximum simultaneous connections (10)
`queueLimit`	how many waiting requests are allowed; `0` = unlimited

The pool is created **once per process**, when the module is first imported. Every controller shares it. Creating a pool *inside* a handler would create ten connections per request and quickly exhaust MySQL's limit — the opposite of the point.

5.3 The import path

From inside `controllers/` , the import must climb one level to reach `src/` :

```
import { sqlConnectionPool } from '../configs/DbConfig.js';
```

File	Path to the config
`index.js` (root)	`./src/configs/DbConfig.js`
`controllers/ProductController.js`	`../configs/DbConfig.js`
`routers/ProductRouter.js`	`../controllers/ProductController.js`

5.4 Why a named export

A config module may legitimately export several things — a pool, an email client, a storage path. Named exports keep each one explicit: `import { sqlConnectionPool }` . A default export would force the importer to invent a name for something that already has one.

> **Pitfall:** file-name casing is part of the path on Linux. The archive has `DbConfig.js`; an import of `dbconfig.js` works on Windows and fails on a Linux server. Match the archive exactly.

5.5 Secrets in real projects

Added in these notes, not written in the lecture: credentials do not belong in a public repository. The same file with environment variables:

```
import { createPool } from "mysql2/promise";

export const sqlConnectionPool = createPool({
    host: process.env.DB_HOST ?? "localhost",
    port: Number(process.env.DB_PORT ?? 3306),
    user: process.env.DB_USER ?? "root",
    password: process.env.DB_PASSWORD ?? "root",
    database: process.env.DB_NAME ?? "productdb",
    connectionLimit: 10,
    queueLimit: 0
});
```

With `node --env-file=.env index.js` (Node 20+), secrets live in a git-ignored `.env`. **No other file changes** — that is the config layer paying for itself.

---

6. The Router Layer and the `export default` Trap

One concern is still in `index.js`: routing. It belongs in `src/routers/`, one router file per entity.

> "Socho, if you are going to have hundreds of routes or hundreds of APIs, to wo saare routes aapne again `index` ke andar hi maintain kiye honge. That thing again should not happen. **Routing should be handled separately by another layer**, jisko aap **router** keh sakte ho."

6.1 What a router is

A **router** is a mini-Express application that only routes: it maps a method + URL to a controller. It does not start a server and does not touch the database.

File	Question it answers
`index.js`	*is there an app, and is it listening?*
`routers/ProductRouter.js`	*which controller handles* `DELETE /product/:id` *?*

File	Question it answers
`controllers/ProductController.js`	*what do I send back?*
`configs/DbConfig.js`	*how do I reach MySQL?*

The naming follows the same per-entity rule: `WelcomeRouter`, `ProductRouter`, `AdminRouter`, later `CustomerRouter`, `CartRouter`, `OrderRouter`.

6.2 `app` does not exist here

> "Do we have the reference of the `app` variable?" — the class answers "*No, sir.*"

Inside `index.js`	Inside `routers/WelcomeRouter.js`
`const app = express()` exists	`app` was never declared
`app.get( ... )` works	`ReferenceError: app is not defined`
full server control	only routing responsibility

A router file is a different module. Nothing from `index.js` is in scope unless it is imported, so Express provides a dedicated factory instead: `Router()`.

6.3 The three-line pattern

```
import { Router } from "express";   // 1. the factory
const someRouter = Router();        // 2. the routing table
someRouter.get("/path", handler);   // 3. routes live on it
```

The methods mirror the HTTP verbs: `.get`, `.post`, `.put`, `.patch`, `.delete`.

6.4 `WelcomeRouter.js`

```
import { Router } from 'express';
import { welcome } from '../controllers/WelcomeController.js'

const welcomeRouter = Router();

welcomeRouter.get("/", welcome);

export default welcomeRouter;
```

`Router()` creates an isolated routing table. `welcomeRouter.get("/", welcome)` uses the same signature as `app.get`, but scoped to this router. `export default` hands the configured table to `index.js`.

The path is `"/"`, not `"/welcome"`, because `index.js` will mount this router at `"/"`. If it were mounted at `"/api"`, the same line would answer `GET /api`. **Mount path lives in the index; relative paths live in the router.**

6.5 `ProductRouter.js`

```
import { Router } from "express";
import {
    save, getAll, getById, deleteById
} from "../controllers/ProductController.js";

const productRouter = Router();

productRouter.post("/product", save);

productRouter.get("/product", getAll);

productRouter.get("/product/:id", getById );

productRouter.delete("/product/:id", deleteById);

export default productRouter;
```

Every second argument is a **reference** to a controller, never a call. The four paths still begin with `/product`, so mounting at `/` keeps the URLs identical.

Method	Path in router	Mount	Final URL	Controller
POST	`/product`	`/`	`POST /product`	`save`
GET	`/product`	`/`	`GET /product`	`getAll`
GET	`/product/:id`	`/`	`GET /product/:id`	`getById`
DELETE	`/product/:id`	`/`	`DELETE /product/:id`	`deleteById`

Mount the same router at `"/api"` and every URL gains the prefix — the router file does not change.

6.6 The `export default` trap

This is the conceptual highlight of the day: a bug caught **before** running the code.

> "**Export kaunse line pe likha hai?** That line number 4. But routing kahan pe define ho rahi hai? At line number 6... Don't you think code ka execution hota hai **line by line**? So six se pehle four chalega. Four pe usne dekha `export` hai, to export kar dega... Export ho raha hai, but **with a blank router object**."

```
// WRONG — export runs before the routes exist
import { Router } from 'express';
import { welcome } from '../controllers/WelcomeController.js';

export default welcomeRouter;        // ❌ the variable does not exist yet

const welcomeRouter = Router();
welcomeRouter.get("/", welcome);
```

Two separate failures:

1. `ReferenceError: Cannot access 'welcomeRouter' before initialization` — with `const`, the name is in the *temporal dead zone* until its declaration executes, and an `export` statement reads the value immediately.
2. Even with `var`, the exported value would be captured before `.get()` ran: an **empty router**. Every request would `404`.

The instructor generalises with a small `demo.js`:

```
// WRONG
export let obj = 0;       // consumers receive 0
obj = obj + 100;          // too late — the value was already handed out
```

```
// RIGHT
let obj = 0;
// ... logic runs ...
obj = 100;
export default obj;       // exported only after the value is final
```

Position of `export default`	Result
before `const router = Router()`	`ReferenceError` (temporal dead zone)
after `Router()` but before the routes	exports an **empty** router → every request `404`
after all routes (bottom)	exports the fully configured router ✅

Rule: declare, configure, then export last. The same rule applies to every router file, including `AdminRouter.js` later.

6.7 Importing a default export

A default export carries no name, so the importer chooses one and **omits the braces**:

```
import welcomeRouter from './src/routers/WelcomeRouter.js';      // ✅
import anythingAtAll from './src/routers/WelcomeRouter.js';      // ✅ legal
import { welcomeRouter } from './src/routers/WelcomeRouter.js';  // ❌ SyntaxError
```

Braces mean "look for a *named* export with this exact identifier". A default export is internally named `default`, so `{ welcomeRouter }` does not exist.

Export style	Export	Import	Name freedom
Named	`export const welcome = …`	`import { welcome } from "…"`	must match (or use `as`)
Default	`export default productRouter`	`import productRouter from "…"`	free choice
Both	one default + named	`import r, { save } from "…"`	default free, named fixed

6.8 Registering the routers

The index file has no idea that router files exist:

> "Toh kya index file ko sapna aayega ki routing ke liye aapne kaunsi file banayi hui hai? Nahi... So from the index file we have to **register our routers**."

```
// index.js — complete final version
import express from "express";
import welcomeRouter from './src/routers/WelcomeRouter.js';
import productRouter from "./src/routers/ProductRouter.js";
import adminRouter from "./src/routers/AdminRouter.js";

const app = express();
app.use(express.json());

app.use("/", welcomeRouter);
app.use("/", productRouter);
app.use("/", adminRouter);
```

```
app.listen(7500);
```

`adminRouter` is added later in the lecture (Part 9); it is shown here so you can see the finished file in one place.

Why `app.use` and not `app.get`: `app.use(path, middleware)` mounts a middleware or router at a path prefix. A router *is* middleware — a function that can inspect a request and either handle it or pass it on. `app.use` therefore means: *for any request whose URL starts with this prefix, let this router try.*

Call	Meaning
`app.get("/", handler)`	only `GET /`
`app.use("/", router)`	every method/path the router knows
`app.use("/api", router)`	same, with `/api` prefixed

6.9 Order matters: `express.json()` first

> "Sabse pehla middleware should be `express.json()`, then after it only we are going to register all the different routers."

Middleware runs in **registration order**. If a router runs first, `request.body` is still `undefined`, destructuring throws, and the `catch` returns a `500` — with the real cause buried in the server console.

6.10 Run and verify

```
node index.js
```

```
GET http://localhost:7500/product
```

```
[
  { "id": 2, "name": "Mouse", "price": 499,
    "description": "Wireless", "qty": 20 },
  { "id": 3, "name": "Keyboard", "price": 6677,
    "description": "Mechanical", "qty": 8 }
]
```

Test	Expected	If it fails
`GET /`	welcome JSON	welcome router not mounted
`GET /product`	array of rows	product router missing, or exported before routes
`POST /product`	`insertedId`	`express.json()` after the routers
`GET /product/1`	one object	route written as `/product` only
`GET /product/999`	`404`	"not found" answered with `500`

Mounting at `"/"` matches every path. Express asks each router in order whether it has a matching route; a router with no match calls `next()` and Express tries the next one. That is why three routers can all be mounted at `/` without conflicting.

6.11 Router errors

Error / symptom	Cause	Fix
`ReferenceError: app is not defined`	used `app.get` in a router file	create a `Router()` object
`Router is not defined`	wrong import	`import { Router } from "express"`
every route `404`	router not mounted, or exported early	`app.use("/", router)` + export last
`Cannot access 'x' before initialization`	`export default` above `const`	move the export to the bottom
`does not provide an export named 'welcomeRouter'`	braces around a default import	remove the braces
`request.body` undefined	`express.json()` registered after routers	register it first
`ERR_MODULE_NOT_FOUND`	missing `.js`, wrong depth/casing	Ctrl+click the path
only `GET /` works	product router never registered	add `app.use("/", productRouter)`

---

7. The Architecture in Motion: Flow, Middleware, Service, DAO

With `index.js` reduced to eleven lines, the instructor draws the whole picture — including the layers that are **not** built today.

7.1 The request flow

```
1. client        →  GET http://localhost:7500/product
2. index.js      →  "a request arrived; let the routers see it"
3. ProductRouter →  "GET /product matches getAll"
4. Controller    →  "SELECT * FROM product through the pool"
5. MySQL         →  returns the rows
6. Controller    →  response.send(rows)
7. client        ←  200 OK [ … ]
```

```
CLIENT ──► index.js ──► ROUTER ──► CONTROLLER ──► MySQL
   ▲                                                 │
   └──────────────────── response ◄──────────────────┘
```

Step	Who decides	Who executes
which layer runs first	Express	`index.js`
which controller for this URL	router	router
what data to fetch and what to send	controller	controller + pool
how to reach MySQL	config	`DbConfig.js`

`index.js` is the **receptionist**; the router is the **directory board**; the controller is the **officer** who hears you and replies; the database is the **records room** only the officer visits.

7.2 Where middleware fits

> "When this middleware has to be executed — after the controller or before the controller? … **Exactly before the controller.** Controller will start processing your request — to wahi to rokna tha."

```
CLIENT ──► index.js ──► ROUTER ──► MIDDLEWARE ──► CONTROLLER ──► DB
```

The middleware is the checkpoint: `next()` continues to the controller; anything else stops the chain with a response.

A middleware has the signature `(request, response, next)`. It either calls `next()` to continue the chain, or sends a response itself and stops everything:

```
function checkSomething(request, response, next) {
    if (/* request is not acceptable */) {
        return response.status(401).send({ message: "Not allowed" }); // stop
    }
    next();   // continue to the controller
}
```

> **Pitfall:** a middleware that neither calls `next()` nor sends a response **hangs the request**. The client waits until it times out. Every path through a middleware must end in one of the two.

`src/middlewares/` exists from the refactor but is **empty today**. The authentication middleware is written in the evening session.

7.3 Service and DAO — the next two layers

The controller still mixes two concerns: **business logic** and **database code**. In a larger application these split again.

Layer	Job	Must not know
Controller	accept the request, send the response	business rules, SQL, that MySQL exists
Service	business rules: validation, calculation, policy	`request` / `response`, which database is used
DAO (Data Access Object)	all database operations	HTTP, business rules

```
CONTROLLER ⟶ SERVICE ⟶ DAO ⟶ DATABASE
```

The result travels back along the same path: DAO → service → controller.

> "Controller ko kuch pata nahi hona chahiye ki jo database — MySQL ya database naam ki koi cheez exist karti bhi hai."

The **factorial example** shows why the service must be reusable:

```
HTTP request  ──► controller ──► service.factorial(5) ──► 120 ──► response
CLI script    ──►            ──► service.factorial(5) ──► 120
Cron job      ──►            ──► service.factorial(5) ──► 120
```

The service takes a number and returns its factorial. It does not know who called it, over HTTP or otherwise — and that ignorance is what lets all three callers share it.

The strongest argument for a DAO is **change isolation**: if MySQL is replaced by PostgreSQL, only the DAO changes. `index.js`, routers, controllers and services stay untouched.

Added in these notes — the shape those layers would take (the lecture describes them but stops at the controller: "abhi sirf controller tak humlog restrict kar rahe hain"):

```
// src/dao/ProductDao.js — only database knowledge
import { sqlConnectionPool } from "../configs/DbConfig.js";

export async function insert(product) {
    const qry =
        `INSERT into product(name, price, description, qty) VALUES(?,?,?,?)`;
    const [result] = await sqlConnectionPool.query(qry,
        [product.name, product.price, product.description, product.qty]);
    return result.insertId;
}

export async function findAll() {
    const [rows] = await sqlConnectionPool.query("SELECT * FROM product");
    return rows;
}
```

```
// src/services/ProductService.js — only business rules
import * as productDao from "../dao/ProductDao.js";

export async function createProduct(input) {
    if (!input.name || input.name.trim().length === 0) {
        throw new Error("Product name is required");
    }
    if (Number(input.price) <= 0) {
        throw new Error("Price must be greater than zero");
    }
    return productDao.insert({ ...input, qty: input.qty ?? 0 });
}
```

The controller then shrinks to a translator: call the service, send its result, turn an exception into a `500`. No SQL, no validation rules, no pool.

7.4 The trade-off

A student asks whether layers increase time complexity. The instructor does not dodge it:

> "Bilkul hota hai increase… **but it is worth it**. Kyunki jitna time aap log development mein waste karoge, ek hi file ke andar sab kuch manage karne mein — that is going to make your application suffer a lot in terms of **scalability**, in terms of **maintainability**… Ya to **architecture se compromise kar lo, ya apni performance se**."

Cost	Benefit
more files, more imports	each file is small and obvious
a few extra function calls per request	independent testing and replacement
more design time up front	parallel work without merge conflicts
slightly more memory/CPU	scale horizontally with cheaper hardware

The extra hops are microseconds; the developer time saved is hours per feature. Runtime cost can be bought back with hardware — a tangled codebase cannot be bought back at any price.

> **Exam:** "Layering adds a negligible constant overhead per request and removes the non-constant cost of changing a growing codebase."

7.5 NestJS, once more

Folder here	Equivalent in NestJS
`routers/`	controller route decorators (`@Get` , `@Post`)
`controllers/`	controller classes
`services/`	providers (`@Injectable`)
`configs/`	`ConfigModule`
`middlewares/`	middleware, guards, interceptors
manual imports	dependency injection

---

8. Securing the APIs: Login Authentication

After the break the lecture changes direction: the code is organised, but every API is still **open**.

8.1 The problem: a URL is a key

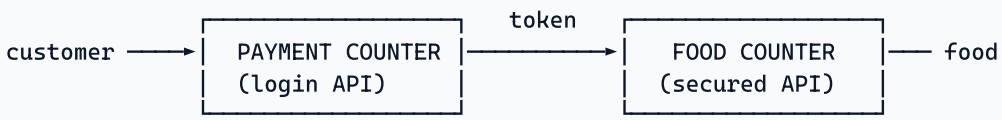
> "Kisi ke bhi paas agar sirf URL haath lag gaya, to **anyone can easily call your API**."

Endpoint	Who can call it now?	Who should?
`POST /product`	anyone	authenticated admin only
`GET /product`	anyone	authenticated user
`GET /product/:id`	anyone	authenticated user
`DELETE /product/:id`	anyone	authenticated admin only

Authentication answers *"who are you?"*; **authorization** answers *"what may you do?"*. Today only authentication is designed.

Term	Question	Example
Authentication	who are you?	login with phone + password
Authorization	what are you allowed to do?	admin may delete, customer may only read
Session	server remembers the login	server-side store, same origin
Token	client proves the login	`Authorization: Bearer …`

8.2 The self-service restaurant analogy



> "What secured resource do they have? **Food is their secured resource.** So to access the food, first you have to make a payment — that is their authentication."

At the food counter they ask for the **receipt**: proof that payment happened. Without it, no food.

Restaurant	Backend
Payment counter	login API (`POST /login`)
Payment	phone + password checked against the DB
Receipt / token	token returned by the login API
Food counter	any secured API
Showing the receipt	client sends the token in the request
Staff checking it	authentication middleware
Lost receipt → pay again	lost token → log in again

8.3 Why not sessions?

A **session** stores login state on the **server**: after login the server remembers you. That works when frontend and backend share an origin.

> "When your frontend is running on a totally different server and your backend is on a totally different server — matlab **origin hi totally different hai** — in that case we can't go with session authentication."

Today React and Express run on different origins, and the backend must be **stateless**: any instance must be able to serve any request.

	Session	Token
State lives	server	client
Across origins	awkward (cookies, CORS, CSRF)	natural
Horizontal scaling	needs a shared session store	any instance can verify
Mobile apps	awkward	natural
Logout	destroy the session	delete the token

The key sentence: the server cannot remember that you logged in. It has no flag, no memory. So the **client carries the proof** — and the proof is the token.

8.4 The login flow

1. The frontend shows a login form (phone + password) and the user clicks **Login**.

2. A request goes to the **login API**.
3. The API reads the credentials from `request.body` and **compares them with the database**.
4. Match → send a success message **and a token**. The message is for the user; the token is the proof.
5. No match → `invalid credentials`. Never reveal whether the user or the password was wrong: that would let an attacker enumerate accounts.

```
POST /login
Content-Type: application/json
```

```
{ "phone": "7070707070", "password": "ramesh123" }
```

```
// 200 OK
{
  "message": "login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 ... "
}
```

```
// 401 Unauthorized
{ "message": "invalid credentials" }
```

8.5 The token and its owner

> "Payment counter ne jo token diya, ab usko sambhal ke rakhne ki responsibility kiska hai — the staff or the customer? **Only the customer is responsible for that token.**"

Restaurant	Backend
customer keeps the receipt	the **client** stores the token
lost receipt → pay again	lost token → log in again
the counter keeps no receipt	the server keeps no session

The client can be a website, an Android app or an iOS app; each stores the token in its own memory (`localStorage` , `sessionStorage` , or an httpOnly cookie) and attaches it to every secured request:

```
// conceptual frontend code — React comes later
const token = localStorage.getItem("token");
```

```
fetch("http://localhost:7500/product/5", {
    method: "DELETE",
    headers: { "Authorization": `Bearer ${token}` }
});
```

Event	What happens
login success	token created and returned
secured request	client attaches the stored token
middleware	checks presence (later: validity and expiry)
token expired	`401` → user logs in again
logout	client deletes the token
token stolen	usable until expiry → short lifetimes and HTTPS

Added in these notes — what the token is (the lecture postpones the format to the evening session): the industry standard is a **JWT**, three Base64URL parts separated by dots.

```
header . payload . signature
eyJhbGciOiJIUzI1NiJ9 . eyJpZCI6MSwicm9sZSI6ImFkbWluIn0 . 4pcPyMD …
```

The payload is only Base64 — **readable by anyone**, so never put a password in it. The **signature** is what the server verifies with a secret key; that is how a token proves identity without the server storing a session.

8.6 The secured-API flow

```
client ⟶ router ⟶ AUTH MIDDLEWARE ⟶ controller ⟶ DB
```

If the token is present and valid, the middleware calls `next()` and the controller runs. If it is missing or invalid, the middleware itself answers `401` and the chain stops — the controller and the database are never reached.

The admin clicks **Delete** on the dashboard. The admin does not know a token exists; the frontend reads it from storage and attaches it. The backend then:

> "First your middleware is going to verify — check if the request is having a token or not... If the request is not having any type of token, then the middleware is itself going to send a response that **token is missing / user is not authenticated**."

Added in these notes — the middleware shape the evening session will implement:

```
// src/middlewares/authMiddleware.js
import jwt from "jsonwebtoken";

export function authenticate(request, response, next) {
    const header = request.headers.authorization;        // "Bearer <token>"
    if (!header) {
        return response.status(401).send({ message: "token missing" });
    }
    const token = header.split(" ")[1];
    try {
        request.user = jwt.verify(token, process.env.JWT_SECRET);
        next();   // continue to the controller
    } catch (error) {
        response.status(401).send({ message: "invalid token" });
    }
}
```

It is attached per route: `productRouter.delete("/product/:id", authenticate, deleteById)`. The controller is then only reachable with a valid token.

8.7 Why a middleware, not the controller?

Two separate questions, two separate answers.

Why is the login check not inside the login API? Because the login API's job is to **create** the proof, not to check it: *"Pehle login to karoge, uske baad check hoga."*

Why not check the token in every controller? Because the middleware already runs before the controller and can read the request: *"controller se pehle kaun chalega? Your middleware... Token mil jata hai to aage controller chala dega."*

	Login API	**Auth middleware**
Purpose	**create** proof: verify credentials, issue token	**check** proof on later requests
Runs	once per login	every secured request
Input	phone + password	token
Output	`{ message, token }` or `invalid credentials`	`next()` or `401`

	Login API	Auth middleware
Knows the DB?	yes, to compare credentials	ideally only verifies the signature

> **Exam:** login **issues** the token, middleware **verifies** the token, the controller does the work.

8.8 Frontend vs backend authentication

> "Dono ek doosre ko correlate karenge. Aapka backend pata hai kis cheez ko secure kar raha hai — **only the APIs**. But frontend kis ko secure karega? Let's say you have a dashboard screen... token nahi hai to user ko dobara login pe **redirect** kar do."

	Backend	Frontend
Protects	the data and the APIs	the screens
Enforced by	middleware verifying the token	route guards checking storage
On failure	`401` JSON	redirect to `/login`
Can the user bypass it?	**no**	**yes**, with dev tools
Token	the same token	the same token

Frontend checks are **UX**; backend checks are **security**. Hiding a delete button protects nothing — anyone can send `DELETE /product/5` with curl.

8.9 Common misunderstandings

Belief	Reality
"login means authentication is done forever"	login issues a proof; every secured request must carry it
"the server remembers me"	it does not — that is why a token exists
"sessions work fine with a React SPA"	possible with cookies + CORS/CSRF work, but tokens are standard for separate origins
"hiding the button secures delete"	no — the API stays public until the backend checks a token
"the login API is the middleware"	login creates the token; middleware checks it

Belief	Reality
"any random string is a token"	it must be verifiable (signature) and expire
"a JWT is encrypted"	the payload is readable; only the signature protects integrity

---

9. Admin Registration and the Password Loophole

The authentication chapter was theory. The last hands-on section builds the first piece: **admin registration**.

> "Login hone se pehle obviously there has to be a **registration** flow... Admin ke liye first we should create a **table** that should be able to store the data for admin."

9.1 The `admin` table

```
CREATE TABLE admin (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(255),
    phone VARCHAR(10),
    password VARCHAR(500)
);
```

Column	Type	Reason
`id`	`INT PRIMARY KEY AUTO_INCREMENT`	surrogate key; MySQL fills it (1, 2, 3...)
`name`	`VARCHAR(255)`	display name
`phone`	`VARCHAR(10)`	the **login identifier** — "based on which admin is going to do a login"
`password`	`VARCHAR(500)`	room for a future **hash** without altering the table

Why `VARCHAR` for a phone and not `INT`? A phone number is an identifier, not a quantity; you never add two of them. Text preserves leading zeros (`0987654321`) and avoids overflow with country codes. `VARCHAR(10)` fits the Indian mobile format used in class, but note: it does **not** enforce "exactly 10 digits". Validation is skipped today ("I'm skipping that part, we will do it later") and belongs in a service layer.

Why 500 characters for a password? A plain password is short, but a bcrypt hash is about 60 characters, and other algorithms need more. Sizing it at 500 now means the column does not change when hashing arrives in the evening.

```
DESC admin;
SELECT * FROM admin;
```

9.2 `AdminController.js`

```
import { sqlConnectionPool } from "../configs/DbConfig.js";

export async function save(request, response) {
    try {
        // step 1: get data from client
        // {name:"", price: 6677, description:"", qty:8}
        const { name, phone, password } = request.body;

        // step 2: create a SQL INSERT query
        // const insertQry = `INSERT into product(name, price, description, qty)
        //   VALUES('${name}', ${price}, '${description}', ${qty})`
        // this above approach (concating data directly in the SQL query)
        // is prone to sql injection attack

        const insertQry = `INSERT into admin(name, phone, password) VALUES(?,?,?)`;

        // step 3: run this query in the db
        const [result] = await sqlConnectionPool.query(
            insertQry,
            [name, phone, password]
        );
        console.log(result.insertId);

        response.send({
            message: "admin registered",
            insertedId: result.insertId
        });

    } catch (error) {
        console.log(error);
        response.status(500).send({ message: "Something went wrong" });
    }
}
```

The structure is deliberately identical to the product `save`. The leftover comments mentioning `product` are also from that copy — harmless here, but a reminder that copy-paste drifts: the comment is now wrong, and the next developer has to check the code to know the truth.

Line / block	Explanation
`import { sqlConnectionPool } …`	**the line that is easy to forget** — and the lecture forgets it once
`export async function save(request, response)`	named export; `async` because of `await`; `save`, not `saveAdmin` (the file already says Admin)
`const { name, phone, password } = request.body`	the three fields the table expects (no `id` — it is auto-incremented)
`VALUES(?,?,?)`	three placeholders for three values
`const [result] = await …query( … )`	`insertId` is the new primary key
`catch`	log the real error, return a generic message

9.3 `AdminRouter.js`

```
import { Router } from "express";
import { save } from "../controllers/AdminController.js";

const adminRouter = Router();

adminRouter.post("/admin", save);

export default adminRouter;
```

`POST /admin`, because creating a resource and carrying a body is what POST is for. The export is at the bottom, after the route — the rule from Part 6. The identifier `save` also exists in `ProductController.js`, but each router imports from its own file, so there is no clash.

9.4 Register it, then test

```
import adminRouter from "./src/routers/AdminRouter.js";
app.use("/", adminRouter);
```

The checklist for every new entity from now on: create the table → create `XxxController.js` → create `XxxRouter.js` → `app.use("/", xxxRouter)` → restart → test in Postman.

```
POST http://localhost:7500/admin
Content-Type: application/json
```

```
{ "name": "Ramesh", "phone": "707070", "password": "ramesh123" }
```

9.5 The live bug: a missing import

First attempt returned:

```
{ "message": "Something went wrong" }
```

Server console:

```
ReferenceError: sqlConnectionPool is not defined
    at save (file:///…/src/controllers/AdminController.js:9:32)
```

`AdminController.js` used `sqlConnectionPool` without importing it. The `try` caught the `ReferenceError`, logged the truth, and sent a generic `500`.

Why this is a good failure: the server did not crash and the client did not hang. It received a finite, structured answer it can display. That is exactly why every handler wraps its body in `try…catch` and answers `500`.

Fix: add `import { sqlConnectionPool } from "../configs/DbConfig.js";`, restart (`node index.js` has no hot reload), and send again:

```
{ "message": "admin registered", "insertedId": 2 }
```

> **Pitfall:** Postman can show a stale response after a fix ("iska caching ka problem hai"). Create a new request or press Send again in a fresh tab.

9.6 Duplicate rows

```
SELECT * FROM admin;
```

```
+----+--------+---------+------------+
| id | name   | phone   | password   |
+----+--------+---------+------------+
|  1 | Ramesh | 707070 | ramesh123 |
|  2 | Ramesh | 707070 | ramesh123 |
+----+--------+---------+------------+
```

> "Wo do baar insert ho gaya hai, kyunki pehle ek baar insert ho gaya tha but data ka operation fail ho gaya tha."

The first attempt had already committed the `INSERT`; only the response failed. The client retried, and the row was inserted again. The instructor cleaned up with `DELETE FROM admin WHERE id = 1`.

Lesson: a failed response does not mean a failed write. In production, guard against this with a unique constraint and/or an idempotency key:

```
ALTER TABLE admin ADD CONSTRAINT uq_admin_phone UNIQUE (phone);
```

Then a retry fails cleanly with `ER_DUP_ENTRY` instead of silently duplicating the admin.

9.7 The loophole: plain-text passwords

> "Aapne password ko aaj **plain text** yahan par store kiya hua hai... Socho, if on your social media applications your passwords get stored as a plain text... **I can easily look into your passwords.**"

Anyone with read access — a DBA, a leaked backup, SQL injection, a log file — sees every password instantly. Because people reuse passwords, one leaked site opens their email and bank too.

Attack	Needs	With plain text
database dump leak	read access	every password known instantly
insider / DBA abuse	one `SELECT`	same
request-body logging	a log file	same
password reuse	one leaked site	the same password elsewhere

> **Pitfall:** passwords must never be stored as plain text. The instructor says "encrypt", which is the everyday word; the technically correct mechanism is a **slow one-way hash** such as bcrypt, Argon2 or scrypt.

	Encryption	Hashing
Reversible?	yes, with the key	no (one-way)
Purpose	protect data you must read later	verify a secret without storing it
For passwords?	❌ a stolen key reveals everything	✅

	Encryption	Hashing
Output	ciphertext	fixed-length digest (`$2b$10$…`)
Login check	decrypt and compare	hash the input, compare digests

Added in these notes — the fix the evening session will implement: hash before inserting, compare on login.

```
// registration: hash BEFORE the insert
import bcrypt from "bcrypt";

const hashedPassword = await bcrypt.hash(password, 10);   // 10 = cost factor
await sqlConnectionPool.query(
    `INSERT into admin(name, phone, password) VALUES(?,?,?)`,
    [name, phone, hashedPassword]
);
```

```
// login: compare the typed password with the stored hash
const [rows] = await sqlConnectionPool.query(
    `SELECT id, name, password FROM admin WHERE phone = ?`, [phone]
);
if (rows.length === 0) {
    return response.status(401).send({ message: "invalid credentials" });
}
const isMatch = await bcrypt.compare(password, rows[0].password);
if (!isMatch) {
    return response.status(401).send({ message: "invalid credentials" });
}
// …create the token here (evening session)
```

Two consequences: the login query must select by `phone` (the identifier), and both "no such admin" and "wrong password" return the same message. Hashing replaces the stored *value*, not the schema — which is why `VARCHAR(500)` was chosen earlier.

9.8 State of the project at the end of the session

Item	Status
Product CRUD, layered	✅ working (`configs` , `controllers` , `routers`)
`index.js`	✅ about 12 lines, no logic
Admin table	✅ created

Item	Status
`POST /admin` registration	✅ working after the import fix
Login API, token creation, auth middleware	⏳ evening session
Password hashing	⏳ evening session — **plain text today, a known bug**
Validation (10-digit phone, duplicates)	⏳ later

---

10. Revision: Tables, Q&A, Practice

10.1 The whole day in ten lines

1. A single-file app works but mixes config, routing, request handling and business logic — fatal at scale.
2. **MVC** splits Model, View and Controller; a backend needs Model + Controller, the View is React.
3. **One file, one job:** `index.js` only starts the server and registers middleware/routers.
4. One **controller** and one **router** per entity; the base URL gets a welcome controller.
5. The pool moves to `configs/DbConfig.js` and is created once, shared by every controller.
6. Routers use `Router()` because `app` does not exist outside `index.js`.
7. `export default router` **goes at the bottom**, after all routes.
8. `app.use("/", router)` mounts routers, always **after** `express.json()`.
9. Auth is **login API** → **token** → **middleware** → **controller**; login issues, middleware verifies.
10. Admin registration works, but the password is plain text — hashing and login come next.

10.2 Definitions

Term	Definition
MVC	design pattern splitting an app into Model, View, Controller
Model	code representation of an entity and its properties
Router	mini-Express object mapping method + URL to a controller
Middleware	`(request, response, next)` function that runs before the controller
Service	business-logic layer, unaware of HTTP and the database engine
DAO	the only layer that talks to the database

Term	Definition
SRP	one module, one reason to change
Connection pool	reusable set of open DB connections
Named export	`export const x` → `import { x }`
Default export	`export default x` → `import anyName`
Authentication	proving who the caller is
Authorization	deciding what that caller may do
Token	verifiable proof of a successful login, stored by the client
Stateless	the server keeps no memory of the client between requests
Hashing	one-way transformation used to store passwords

10.3 Syntax cheat sheet

Need	Syntax
create app	`const app = express()`
parse JSON	`app.use(express.json())`
mount router	`app.use("/", router)`
start server	`app.listen(7500)`
create router	`const r = Router()`
named import/export	`import { save } from "./x.js"` / `export function save() {}`
default import/export	`import router from "./x.js"` / `export default router`
query with values	`await pool.query("… WHERE id = ?", [id])`
read input	`request.body` / `request.params` / `request.headers`
send	`response.send(obj)` / `response.status(500).send(obj)`
middleware	`(request, response, next) ⇒ { … next() }`

10.4 Comparisons

Layer	Responsibility	Knows the DB?
`index.js`	start server, mount middleware/routers	no
Router	URL + method → controller	no
Middleware	inspect/validate before the controller	usually no
Controller	accept request, send response	**no**
Service	business rules	no
DAO	database operations	**yes**

	`app`	`router`
created by	`express()`	`Router()`
lives in	`index.js`	`routers/*.js`
can listen	yes	no
mounted with	—	`app.use("/", router)`
purpose	the server	a routing table

	Login API	Auth middleware	Password: plain	Password: hashed
job	issue token	verify token	readable by a DBA	unreadable
runs	once per login	every secured request	reversible	one-way
failure	`invalid credentials`	`401`, controller never runs	full account takeover	no leak
check	DB compare	signature/expiry	string compare	`bcrypt.compare`

10.5 Common mistakes

Mistake	Symptom	Fix
`app.get("/", welcome())`	crash at startup	pass `welcome`

Mistake	Symptom	Fix
forgetting `export`	"does not provide an export named…"	export the function
braces on a default import	`SyntaxError`	remove the braces
`export default` before routes	every request `404`	export at the bottom
`app.get` in a router file	`app is not defined`	`const r = Router()`
router never mounted	all routes `404`	`app.use("/", router)`
`express.json()` after routers	`request.body` undefined → `500`	register it first
missing `.js` / wrong `../`	`ERR_MODULE_NOT_FOUND`	correct the path
missing pool import	`500` + `ReferenceError`	import from `../configs/DbConfig.js`
`await` without `async`	`SyntaxError`	add `async`
SQL by concatenation	SQL injection	`?` + values array
plain-text password	breach on leak	hash before insert
assuming a `500` means "no insert"	duplicate rows	unique constraint / idempotency
middleware without `next()`	request hangs	call `next()` or send a response

10.6 Interview questions

1. **Why not keep all APIs in `index.js`?** Hundreds of APIs and four unrelated concerns in one file become unscalable, hard to maintain and hard to debug, even though it runs.
2. **What is MVC, and which part is missing in a backend?** Model–View–Controller; the View is missing — the API returns JSON and React draws the UI.
3. **What is a controller responsible for?** Accepting the request and generating the response. Nothing else.
4. **Why `Router()` instead of `app` in a router file?** `app` is scoped to `index.js`; `Router()` creates an isolated routing table mounted with `app.use`.
5. **Why must `export default router` be last?** Modules execute top to bottom; exporting first exports an empty router (and with `const`, throws a TDZ error).

6. **Named vs default export?** Named: braces, exact name, many per file. Default: no braces, any name, one per file.
7. **How does token authentication work end to end?** Login returns a token → the client stores it and attaches it to secured calls → middleware verifies it → the controller executes or the server returns `401`.
8. **Why not sessions?** Frontend and backend are on different origins and the server is stateless; a token is carried by the client and verifiable by any instance.
9. **Where should validation live?** In the service layer (or validation middleware), not in the controller.
10. **Why check the token on the backend if the frontend hides the buttons?** Frontend checks are UX and can be bypassed; only server checks protect data.
11. **Why is `VARCHAR(500)` used for a short password?** To hold a future hash without altering the table.
12. **Why is a phone stored as `VARCHAR`, not `INT`?** It is an identifier; text preserves leading zeros and country codes.
13. **What does layering cost?** A few function calls per request — negligible next to the maintenance cost it removes.
14. **How would you add a `customer` entity?** Table, `CustomerController.js`, `CustomerRouter.js`, one `app.use` line; nothing else changes.
15. **What changes if MySQL becomes PostgreSQL?** Only the DAO (and driver/config); every other layer stays.

10.7 Practice, easy to hard

1. Rebuild the project: `npm init`, install `express` and `mysql2`, set `"type": "module"`, create `index.js`, `DbConfig.js`, two controllers and two routers.
2. Prove all five product endpoints in Postman.
3. Break two things on purpose — remove `express.json()`; move `export default` to the top of `WelcomeRouter.js` — record each symptom and fix it.
4. Add `PUT /product/:id` in the existing style.
5. Recreate the `admin` table and the registration API from memory.
6. Move all SQL out of `ProductController` into a DAO and all validation into a service; the controller must contain neither.
7. Design the login API on paper: the query, both responses, where the token is created, and where it is checked.
8. Write an auth middleware signature and list every response it can produce.

10.8 Final checklist

- [ ] Explain why a working single-file app is still a bad design.
- [ ] Draw the MVC split and the final folder tree from memory.
- [ ] Write a controller with `async`, `try/catch`, placeholders and a `500` fallback.
- [ ] Explain why the pool lives in `configs` and is created once.
- [ ] Write `Router()` boilerplate, including the export at the bottom.
- [ ] Explain the `export default` timing bug with the `obj = 100` example.
- [ ] Explain why `express.json()` precedes the routers.
- [ ] Trace a request through index → router → middleware → controller → DB.
- [ ] Explain the service/DAO layers and the database-swap benefit.
- [ ] Describe login → token → middleware → controller with the restaurant analogy.
- [ ] Explain why the backend check is mandatory and the frontend check is only UX.
- [ ] Write `CREATE TABLE admin` and justify every type.
- [ ] Reproduce the admin registration API and the missing-import bug.
- [ ] Explain the plain-text password loophole and what hashing changes.

---

11. Run-Book: Commands, Postman, Endpoints

11.1 Commands

Command	Purpose
`npm init -y`	create `package.json`
`npm pkg set type=module`	enable `import` / `export`
`npm i express mysql2`	install server + MySQL driver into `node_modules`
`node index.js`	start on port 7500
`node --watch index.js`	restart automatically on change (optional)

`package.json` must contain `"type": "module"`, or `import express from "express"` throws `SyntaxError: Cannot use import statement outside a module`. `node_modules` is recreated by `npm i` and never committed.

Completed layout:

```
crud-apis/
├── index.js                    ← app · express.json · routers · listen
├── package.json                ← type: module, express, mysql2
└── src/
    ├── configs/
    │   └── DbConfig.js         ← createPool → sqlConnectionPool
    ├── controllers/
    │   ├── WelcomeController.js ← welcome
    │   ├── ProductController.js ← save, getAll, getById, deleteById
    │   └── AdminController.js   ← save
    ├── middlewares/            ← empty today (auth comes in the evening)
    └── routers/
        ├── WelcomeRouter.js    ← GET /
        ├── ProductRouter.js    ← /product endpoints
        └── AdminRouter.js      ← POST /admin
```

11.2 Postman requests

```
POST http://localhost:7500/product
Content-Type: application/json

{
  "name": "Keyboard",
  "price": 6677,
  "description": "Mechanical keyboard",
  "qty": 8
}
```

```
{ "message": "product inserted", "insertedId": 1 }
```

```
POST http://localhost:7500/admin
Content-Type: application/json

{ "name": "Ramesh", "phone": "707070", "password": "ramesh123" }
```

```
{ "message": "admin registered", "insertedId": 2 }
```

The body type must be **raw** → **JSON**; otherwise `request.body` is `undefined`.

11.3 Endpoint inventory

Method	URL	Layer	Status
GET	`/`	`WelcomeRouter` → `WelcomeController`	public
POST	`/product`	`ProductRouter` → `save`	should be secured

Method	URL	Layer	Status
GET	`/product`	`ProductRouter` → `getAll`	should be secured
GET	`/product/:id`	`ProductRouter` → `getById`	should be secured
DELETE	`/product/:id`	`ProductRouter` → `deleteById`	should be secured
POST	`/admin`	`AdminRouter` → `save`	registration, stays public
POST	`/login`	*(evening session)*	issues the token

11.4 What comes next

Next step	Content
Homework	rebuild the layered project; admin table + registration API
Evening session	login API, token creation, auth middleware, password hashing
Later	React frontend, frontend route guards, full service/DAO layers
Self-study	NestJS — the same architecture out of the box

11.5 If you remember only five things

1. A file that works is not a file that scales. Separate configuration, routing, request handling and business logic.
2. `index.js` receives, the router chooses, the controller answers, the config owns the pool.
3. Configure first, `export default` last — otherwise you export an empty router.
4. `express.json()` before the routers; middleware runs in the order it is registered.
5. Login issues a token, middleware verifies it, and passwords are hashed — never stored as plain text.

The fastest way to make this permanent is to rebuild the project from an empty folder and break it deliberately at each boundary — remove an export, mount a router twice, drop the parser — until the error messages become familiar.